

the AtomicResponse SHALL be FAILURE.

- i. The server SHALL NOT associate an [Atomic Write State](#) with the client.
 - ii. Depending on the [status codes](#) received for individual attributes, the client MAY choose to try again; e.g. if the attribute the client was attempting to write to has status BUSY, the client could attempt to write to it again after a pause.
- e. Otherwise, the [StatusCode](#) SHALL be SUCCESS.
- i. The server SHALL create an [Atomic Write State](#) with the endpoint number and ID of the cluster instance handling the AtomicRequest command, the [accessing fabric](#), the [Atomic Writer ID](#), and the set of attrib-ids returned.
 - ii. If the requested timeout is longer than the maximum timeout the server allows for the requested set of attributes, the server MAY return the maximum timeout for the requested set of attributes in the [Timeout](#) field. Otherwise, the server SHALL return the requested timeout in the [Timeout](#) field.
 - iii. If the server does not receive a matching AtomicRequest with a RequestType of CommitWrite from the associated client before the timeout provided by the server in the AtomicResponse, the server SHALL roll back any pending writes and discard the atomic write.

Commit Write

If the RequestType of the AtomicRequest is CommitWrite, the server SHALL attempt to commit any pending writes and return an AtomicResponse:

1. If the client is not associated with an [Atomic Write State](#) whose endpoint-no and cluster-id match the endpoint number and ID of the cluster instance handling the request, and whose set of attrib-ids match the attrib-ids requested, irrespective of order, the server SHALL return an error code of INVALID_IN_STATE.
2. The server SHALL process all pending writes to the requested atomic attributes as if they had arrived in a single message.
3. The server SHALL create a list of [attribute statuses](#) for each attribute that would be modified by the processing of the atomic write:
 - a. If the server is able to determine that the atomic write will not succeed, the [status code](#) SHALL indicate the error code that would have been received on the first failed write to that attribute.
 - i. For example, if a write is an attempt to replace a list, but the data version has changed since the atomic write began due to a command invocation, the [status code](#) would be DATA_VERSION_MISMATCH.
 - b. Otherwise, the [status code](#) SHALL be SUCCESS.
4. If the [status code](#) of any [AtomicAttributeStatusStruct](#) in the list is not SUCCESS, then the server SHALL:
 - a. Discard all pending writes to the attributes associated with the [Atomic Write State](#) associated with the client
 - b. Discard the [Atomic Write State](#) associated with the client